Edge cases (a missing required argument) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `missing_argument_practice.py`: when a required argument is missing from the user's message, the model can ask, guess, or send an empty value — and which one it does tells you whether your tool needs to design for a clarifying question or a graceful default. **If not:** the Build Task's Pydantic argument models would be guessing at a problem you'd never actually watched happen.

Solution

```
# missing_argument_practice.py
from langchain_core.runnables import Runnable
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a named city. Requires a city name."""
    return f"Sunny in {city}"

def get_model_with_tools() -> Runnable:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return model.bind_tools([get_weather])

def classify_response(response) -> str:
    # why: sorts each run into one of 3 real outcomes instead of leaving it
    # to a human reading scrolled-past terminal text.
    if not response.tool_calls:
        return "asked_or_answered_directly"
    city_sent = response.tool_calls[0]["args"].get("city")
    if not city_sent:
        return "called_with_empty_value"
    return "called_with_guessed_value"

def run_missing_argument_trial(prompt: str, runs: int = 5) -> list[dict]:
    model_with_tools = get_model_with_tools()
```

```
    records = []

    for run_number in range(1, runs + 1):
        response = model_with_tools.invoke(prompt)
        outcome = classify_response(response)
        record = {"run": run_number, "outcome": outcome}
        if outcome == "called_with_guessed_value":
            record["city_sent"] = response.tool_calls[0]["args"].get("city")
        if outcome == "asked_or_answered_directly":
            record["reply"] = response.content
        records.append(record)

    return records

if __name__ == "__main__":
    results = run_missing_argument_trial("what's the weather like")
    for record in results:
        print(record)
```

Expected output (example):

```
{'run': 1, 'outcome': 'asked_or_answered_directly', 'reply': 'Which city?'}
{'run': 2, 'outcome': 'called_with_guessed_value', 'city_sent': 'London'}
{'run': 3, 'outcome': 'asked_or_answered_directly', 'reply': 'Specify a city?'}
{'run': 4, 'outcome': 'called_with_guessed_value', 'city_sent': 'New York'}
{'run': 5, 'outcome': 'called_with_empty_value'}
```